EXPERIMENT – 1

Comparative Study of Low-Code Development versus Traditional Application Development

Department of Computer Science and Engineering
NAME: Pranav Narayanan.T
ROLL NO: 240701393

AIM

To compare low-code and traditional application development by examining their development stages, workflow, and key technical trade-offs.

INTRODUCTION TO TRADITIONAL APPROACH

Traditional application development involves implementing application logic directly through programming. Developers use code editors, compilers, and integrated development environments to design databases, connect APIs, and manage deployment. This method provides strong control over the architecture and source code, but usually requires more development time, higher costs, and extensive testing.

INTRODUCTION TO LOW-CODE APPROACH

Low-code platforms use visual development tools, reusable components, templates, and automated deployment features to reduce manual coding. They allow both professional developers and citizen developers to build applications more quickly. This can reduce initial cost and delivery time, although customization may be limited by the platform and the solution may depend on its ecosystem.

STAGES IN TRADITIONAL APPROACH

The traditional Software Development Life Cycle follows a structured sequence of engineering activities:

1. Requirement Analysis
2. Architecture and Design
3. Manual Development
4. Quality Assurance
5. System Integration
6. Production Deployment
7. Ongoing Support



24070157

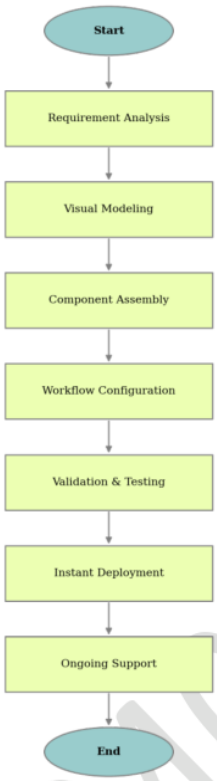
Stages in Low-Code Approach

The low-code workflow swaps out manual coding stages for configuration-driven steps, each needing far less hands-on engineering:

STAGES IN LOW-CODE APPROACH

The low-code process emphasizes visual modeling and configuration instead of building every component manually:

1. Requirement Analysis
2. Visual Modeling
3. Component Assembly
4. Workflow Configuration
5. Validation and Testing
6. Instant Deployment
7. Ongoing Support



24070173

Comparative Analysis

Feature	Traditional Development	Low-Code Development
Development Speed	Slower, as every module is hand-built and individually tested	Considerably faster, since ready-made components cut build time sharply
Skill Required	Demands trained developers fluent in programming languages and frameworks	Minimal coding skill needed; drag-and-drop tools suit citizen developers

COMPARATIVE ANALYSIS

Feature	Traditional Development	Low-Code Development
Development Speed	Slower, as each module is built and tested individually	Faster because ready-made components reduce build time
Skill Required	Requires trained developers familiar with programming languages and frameworks	Requires less coding; visual tools can support citizen developers
Customization	Offers broad control over code and architecture	Limited by available platform components and connectors
Cost	Higher initial investment in developer time and infrastructure	Lower initial cost, with possible ongoing subscription or licensing fees
Maintenance & Updates	Fixes and enhancements usually require code changes and re-testing	Platform updates may be delivered centrally with less manual effort
Vendor Lock-in	Codebase ownership provides more independence from a platform vendor	Applications can become dependent on the chosen platform ecosystem
Best Use Cases	Complex, performance-critical, or security-sensitive systems	Rapid prototypes, internal tools, and workflow automation

RESULT

The comparison indicates that traditional development is better suited to complex or highly customized systems where direct control is essential. Low-code is often a faster and more economical option for common business applications and workflow automation. Organizations may combine both approaches, using low-code for interfaces and workflows while keeping demanding backend components in traditional code.

Original screenshots from the supplied experiment

Experiment – 1

Comparative Study of Low-Code Development versus Traditional Application Development

Aim

To examine low-code and traditional application development side by side, focusing on their development stages, workflow style, and the underlying technical trade-offs of each approach.

Introduction to Traditional Approach

Traditional development relies on writing every piece of application logic directly in code, using conventional editors, compilers, and IDEs. Engineers design database schemas, wire up APIs, and manage deployment infrastructure themselves, drawing on strong command of languages such as Java, Python, or C++. This hands-on approach grants complete authority over the codebase and system architecture, though it typically demands longer build times, larger budgets, and thorough testing cycles.

Introduction to Low-Code Approach

Low-Code Application Platforms replace much of the manual coding effort with cloud-hosted, visual builders. Developers assemble applications through drag-and-drop components, ready-made templates, and automated release pipelines, which lowers the barrier enough for "citizen developers" with little programming background to ship working software. By automating routine programming work, low-code speeds up delivery and trims upfront cost, at the expense of deep customization and some dependency on the chosen platform.

Stages in Traditional Approach

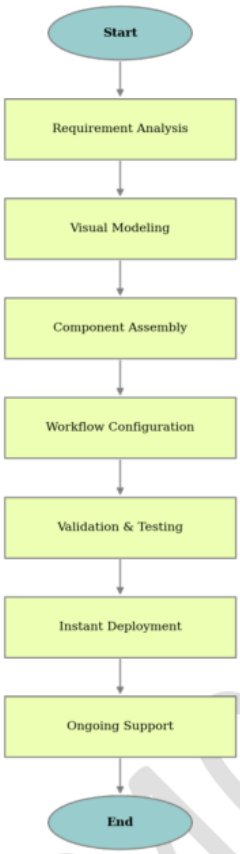
The traditional Software Development Life Cycle moves through a structured, engineering-heavy sequence, with dedicated effort required at every stage:



2407015

Stages in Low-Code Approach

The low-code workflow swaps out manual coding stages for configuration-driven steps, each needing far less hands-on engineering:



2407O1573

Comparative Analysis

Feature	Traditional Development	Low-Code Development
Development Speed	Slower, as every module is hand-built and individually tested	Considerably faster, since ready-made components cut build time sharply
Skill Required	Demands trained developers fluent in programming languages and frameworks	Minimal coding skill needed; drag-and-drop tools suit citizen developers

Customization	Unrestricted — every line of logic can be shaped as needed	Bounded by the platform's available components and connectors
Cost	Larger upfront investment in developer time and infrastructure	Lower initial spend, offset by ongoing subscription or licensing fees
Maintenance & Updates	Every fix or enhancement requires manual code changes and re-testing	Platform vendor pushes updates centrally with little manual effort
Vendor Lock-in	Absent — the organization retains full ownership of the codebase	Present — the application is tied to the platform's ecosystem
Best Use Cases	Large-scale, performance-critical, or security-sensitive systems	Rapid prototyping, internal tools, and workflow automation

Result

The comparison confirms that traditional development remains the stronger choice for complex, high-performance, or tightly regulated systems, where full control matters most. Low-code, on the other hand, offers a quicker and more economical route for everyday business applications. Many organizations now blend the two — leaning on low-code for front-end and workflow needs while reserving traditional development for demanding back-end requirements.

240701573

ROLL NO : 240701573

NAME : UMAR FAROOK K